How-To Library

Informatica

Defining Table Properties in a Data Map to Normalize an Array

© Copyright Informatica LLC 2010, 2021. Informatica

Abstract

This article describes how to define table properties in a data map to normalize a field defined as an array with multiple occurrences.

Supported Versions

- PowerExchange 10.0 and later

Table of Contents

Overview. 2
Defining Table Properties in a Data Map to Normalize an Array - Example 2
Step 1. Add a Data Map that Contains an Array Defined with Multiple Occurrences. 3
Step 2. Add User-Defined Fields that Demonstrate the Normalization of an Array. 4
Step 3. Define Table Properties to Normalize an Array. 5
Step 4. Test the Normalization of the Array in the Data Map. 6
Reference Information for PowerExchange Functions. 6
GenVRowKey. 6
GetSeqWithinLevel. 7
Table Properties - Definition. 7

Overview

This article describes how to define table properties in a data map that contains an array with multiple occurrences. The table properties normalize the array and produce the correct output from the source file. This article demonstrates how to complete the following tasks:

- Add a data map that contains an array with multiple occurrences.
- Add user-defined fields to the data map. These user-defined fields invoke the PowerExchange GenVRowKey and GetSeqWithinLevel functions, which demonstrate the normalization of the array in the database row test output.
- Define table properties that normalize the array in the database row test output.

For more information about using PowerExchange functions in user-defined fields to process data in data map records, see the *PowerExchange Navigator User Guide*.

Defining Table Properties in a Data Map to Normalize an Array - Example

To define the layout of multiple records one time rather than multiple times, define an array with multiple occurrences. Then, define table properties in the data map to normalize the array and produce the correct output from the source file.

In this example, you add user-defined fields to the data map. These fields invoke the PowerExchange GenVRowKey and GetSeqWithinLevel functions, which demonstrate the normalization of the array in the database row test output.

This example shows how to complete the following tasks:

1. Add a data map for a sequential flat file.

   When you add the data map, you import a COBOL copybook that defines an array with multiple occurrences into the data map.

2. Add user-defined fields to the data map.

   These user-defined fields invoke the PowerExchange GenVRowKey and GetSeqWithinLevel functions, which demonstrate the normalization of the array in the database row test output.

   For a source row that contains an array, the GenVRowKey function generates multiple table rows identified by unique row numbers.

   The GetSeqWithinLevel function gets the sequence number of a record in the current hierarchical level.

3. Define table properties that normalize the array in the database row test output.

4. Run a database row test to display the results of normalization.

   In the database row test output, one row appears for each occurrence in the array.

In this example, you use the demo2.dat data file and the demo2.cob copybook to create the demo.map2array data map.

The demo2.cob copybook uses the following OCCURS clause to define an array with multiple occurrences:

```
04  ITEM OCCURS 3 DEPENDING ON ITEMCT PIC X(15).
```

Step 1. Add a Data Map that Contains an Array Defined with Multiple Occurrences

In this step, you add a data map for a sequential flat file and import a COBOL copybook that defines an array with multiple occurrences into the data map.

1. On the **Resources** tab in the **Resource Explorer**, click **Add** > **Data Map**.
2. In the **Name** dialog box, enter the following information:
   - In the **Schema Name** box, enter **demo**.
   - In the **Data Map Name** box, enter **map2array**.
   - Select the **Import Record Definitions** option.
   - In the **Access Method** list, select **SEQ**.
3. Click **Next**.
4. In the **SEQ Access Method** dialog box, enter the following information:
   - In the **File Name** box, click the Browse button to browse to the demo2.dat file in the examples directory.
   - In the **Record Format** area, select the **Field Separator** option and enter the comma (,) character in the **Field Separator** box.

     The demo2.dat data file is a comma (,) separated file.
5. Click **Finish**.
6. In the **Import Copybook - Source Details** dialog box, enter the following information:
   - In the **Source** area, select **Local**.
   - In the **Type** list, select **COBOL**.
   - In the **Column Range** area, enter **7** in the **Start** box and **72** in the **End** box.
7. Click **Next**.

8. In the **Import Copybook - Local Cobol Details** dialog box, enter the following information:
   - Click the Browse button to browse to the demo2.cob file in the examples directory.
   - To view the copybook, click **Preview**. The copybook includes the following OCCURS clause to define an array with multiple occurrences:

     ```
     04  ITEM OCCURS 3 DEPENDING ON ITEMCT PIC X(15).
     ```
9. Close the preview window. In the **Import Copybook - Local Cobol Details** dialog box, click **Next**.
10. In the **Import Copybook - Configuration Details** dialog box, review the selected actions for imported records, fields, and tables and click **Finish**.
11. In the **Import Copybook Information** window, review the information for the import and click **OK**.
12. To import the RECORD_IN record, click **OK** in the **Record Definition** dialog box.
13. To import the fields in the record, click **OK** in the **Field Definition** dialog box for each field.
14. To import the RECORD_IN table, click **OK** in the **Table Definition** dialog box.

    The **Cobol Import** window displays the imported copybook. The **Copybook Message Log** window displays the results of the import operation.
15. Close the **Cobol Import** and the **Copybook Message Log** windows.
16. On the **Resources** tab in the **Resource Explorer**, select the RECORD_IN table and click **File** > **Database Row Test**.

    The PowerExchange Navigator displays an error message that indicates that the RECTYPE field in the RECORD_IN record is an invalid type for a delimited data file. Additionally, the ITEMCT field is an invalid type.
17. Change the datatype of the RECTYPE and ITEMCT fields by completing the following actions:
    - In the **Record** window for the RECORD_IN record, double-click the RECTYPE field.

      In the **Field Properties** dialog box, select a **Field Type** of NUMCHAR. Enter 2 in the **Length** box. Click **OK.**
    - In the **Record** window, double-click the ITEMCT field.

      In the **Field Properties** dialog box, select a **Field Type** of NUMCHAR. Enter 1 in the **Length** box. Click **OK.**
18. On the **Resources** tab in the **Resource Explorer**, select the RECORD_IN table and click **File** > **Database Row Test**.
19. On the **Database Row Test** dialog box, click **Go**.

    The **Database Row Test Output** window displays the results of the database row test.

Step 2. Add User-Defined Fields that Demonstrate the Normalization of an Array

In this step, you add user-defined fields that invoke PowerExchange functions that demonstrate the normalization an array with multiple occurrences.

The user-defined fields invoke the PowerExchange GenVRowKey and GetSeqWithinLevel functions. These functions add data to the database row test output that demonstrate how the table properties that you defined normalize the array. When you run a database row test, PowerExchange displays one row for each occurrence in the array. For more information about these functions, see "GenVRowKey" on page 6 and "GetSeqWithinLevel" on page 7.

1. Open the demo.map2array data map and the RECORD_IN record.
2. In the **Record** window, click the **Expr(0)** tab.
3. Right-click anywhere on the **Expr(0)** tab and click **Add Field at End**.
4. In the cell in the **Name** column, enter `r1genvrow_idx`.
5. In the cell in the **Type** column, select **NUM32U**.

6. In the cell in the **Phase** column, select **R**.

   This selection indicates a read operation.
7. To enter an expression for the field, click in the cell in the **Expression** column and click the Browse button.

   The **Expression Editor** dialog box appears.
8. In the **Function List** list in the **Expression Editor** dialog box, double-click the GenVRowKey function.
9. In the **Expression List** list, enter `()` at the end of the GenVRowKey function name. Click **Validate**.

   In the **Validate** box, the `No Errors` message appears.
10. Click **OK**.
11. Right-click anywhere on the **Expr(0)** tab and click **Add Field at End**.
12. In the cell in the **Name** column, enter `r1genseq`.
13. In the cell in the **Type** column, select **NUM32**.
14. In the cell in the **Phase** column, select **R**.

    This selection indicates a read operation.
15. To enter an expression for the field, click in the cell in the **Expression** column and click the Browse button.

    The **Expression Editor** dialog box appears.
16. In the **Function List** list in the **Expression Editor** dialog box, double-click the GenSeqWithinLevel function.
17. In the **Expression List** list, enter `()` at the end of the GenSeqWithinLevel function name. Click **Validate**.

    In the **Validate** box, the `No Errors` message appears.
18. Click **OK**.

Step 3. Define Table Properties to Normalize an Array

In this step, you define table properties that normalize an array in a data map.

1. Open the demo.map2array data map.
2. On the **Data Map** tab in the **Resource Explorer**, right-click the RECORD_IN table and click **Properties**.

   The **Table Properties - Definition** dialog box appears.
3. In the **Column Generation** list, select **Refresh with missing columns**.

   Because the record on which the table is based contains new fields, corresponding columns are added to the table.
4. Click **OK**.
5. On the **Data Map** tab in the **Resource Explorer**, right-click the RECORD_IN table and click **Properties**.

   The **Table Properties - Definition** dialog box appears.
6. In the **Table Properties - Definition** dialog box, complete the following actions:
   - In the **Column Generation** list, select **Apply array format changes**.

     Because the record on which the table is based contains changed fields that are defined as arrays, those changes are reflected in the corresponding columns in the table.
   - In the **How do you want to handle multiple instances of selected record** list, select **New Row**.

     This action causes PowerExchange to display a new row for each occurrence in the array in a database row test.

7. Click **OK**.

   For more information about the **Table Properties - Definition** dialog box, see "Table Properties - Definition" on page 7.

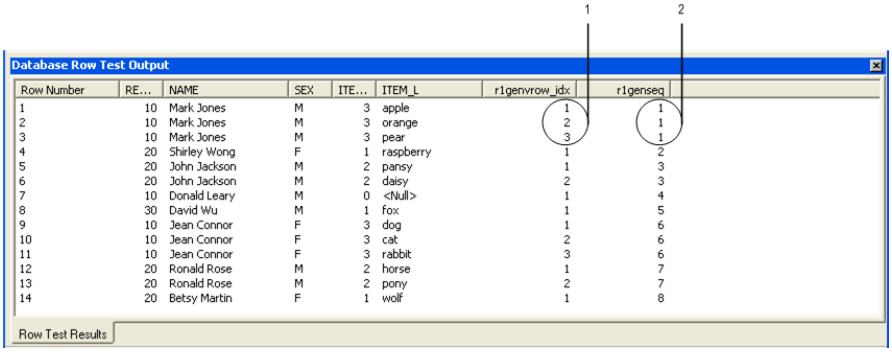
Step 4. Test the Normalization of the Array in the Data Map

In this step, you run a database row test to test the normalization of the array in the data map.

1. Open the demo.map2array data map.
2. On the **Data Map** tab in the **Resource Explorer**, select the RECORD_IN table and click **File** > **Database Row Test**.

   The **Database Row Test** dialog box appears.
3. In the **Database Row Test** dialog box, click **Go**.

   The **Database Row Test Output** window displays the following results:



Row Number	RE...	NAME	SEX	ITE...	ITEM_L	r1genvrow_idx	r1genseq
1	10	Mark Jones	M	3	apple	1	1
2	10	Mark Jones	M	3	orange	2	1
3	10	Mark Jones	M	3	pear	3	1
4	20	Shirley Wong	F	1	raspberry	1	2
5	20	John Jackson	M	2	pansy	1	3
6	20	John Jackson	M	2	daisy	2	3
7	10	Donald Leary	M	0	<Null>	1	4
8	30	David Wu	M	1	fox	1	5
9	10	Jean Connor	F	3	dog	1	6
10	10	Jean Connor	F	3	cat	2	6
11	10	Jean Connor	F	3	rabbit	3	6
12	20	Ronald Rose	M	2	horse	1	7
13	20	Ronald Rose	M	2	pony	2	7
14	20	Betsy Martin	F	1	wolf	1	8

1. Results of the GenVRowKey function
2. Results of the GetSeqWithinLevel function

   The r1genvrow_idx column displays the results of the GenVRowKey function, which is the number of the occurrence in the array. The r1genseq column displays the results of the GetSeqWithinLevel function, which is the number of the record.

   For example, the following results appear in the output:

   - The first row displays data for occurrence 1 in the array in record 1.
   - The third row displays data for occurrence 3 in the array in record 1.
   - The fourth row displays data for occurrence 1 in the array in record 2.

Reference Information for PowerExchange Functions

Use the following information to include the GenVRowKey and GetSeqWithinLevel functions in user-defined fields in data maps.

GenVRowKey

Generates multiple table rows identified by unique row numbers from a row created from a source row that contains an array.

This function enables you to differentiate among multiple target rows created by a single source row. For example, if a source file contains an array and a child table is created from the array, use this function to populate a target field with unique multiple table rows for rows that would otherwise be identical.

Note: For data rows with an OCCURS clause, a single row is returned for each instance of the OCCURS.

Syntax:

```
result=GenVRowKey()
```

Returns:

NUM32U. Multiple unique table rows for a row created from a source row that contains an array.

GetSeqWithinLevel

Sequential flat files, IMS, and IDMS. Gets the sequence number of a record in the current hierarchical level.

Syntax:

```
result=GetSeqWithinLevel()
```

Returns:

NUM32. The record sequence number.

Table Properties - Definition

View or edit table definition properties.

Available Records

A list of the records in the data map that are not in the complex table.

To add a record to the **Record Dependencies** list, in the **Available Records** list, right-click a record and click **Add Record**.

To add a child record to a parent record:

- In the **Record Dependencies** list, select a record to identify it as the parent record.
- In the **Available Records** list, right-click a record and click **Add Record as Child**. This action moves the record to the **Record Dependencies** list as a child record of the parent record.

For IDMS: To select the system index to use for record retrieval, right-click a record and click **Use System Index**. Then, select the index.

Record Dependencies

A list of the records that are in the complex table, with any defined hierarchical dependencies.

To remove a record dependency, right-click a record and click **Delete**.

For IDMS, to reverse the direction of the area read, right-click a record and click **Reverse Area Read**. To reverse the direction of the set read, right-click the record and click **Reverse Set Read**.

How do you want to handle multiple instances of selected records?

Select one of the following options:

- **New Row**. A new row appears or is written to the target for every instance of the record or segment.
- **Ignore**. Second and subsequent instances of a record or segment appear or are written to the target.
- **Array**. The number of records or segments specified in the **Array** list appear or are written to the target in a single row of output.

PowerExchange populates the output row until it is full, and then completes one of the following actions:

- If you clear the **New Row on Overflow** option, PowerExchange ignores subsequent records or segments.
- If you select the **New Row on Overflow** option, PowerExchange displays a new row with the overflow records or segments.

For example, for a record with five instances, if you enter 3 in the **Array** list, PowerExchange builds two output rows. The first row contains an array of three instances, and the second row contains an array of two instances.

Note: If you set a parent record or segment to **Array**, you must set all child records or segments to **Ignore**.

Column Generation

When you first define a table, PowerExchange derives column names from the field names in the record on which the table is based. However, PowerExchange uses a special naming convention for records that contain fields defined as arrays.

Select one of the following options to indicate how to refresh columns in a table after you change field definitions in the record on which the table is based:

- **Apply array format changes**. If the record on which the table is based contains changed fields that defined as arrays, those changes are reflected in the corresponding columns in the table.
- **Refresh with missing columns**. If the record on which the table is based contains new fields, corresponding columns are added to the table.
- **Reset to defaults**. PowerExchange resets column names to the corresponding fields names in the record on which the table is based. PowerExchange discards any changes that you made to column names in the table.
- **Remove Hidden Columns**. For DB2UNLD. PowerExchange generates a with the default values of **Hide from Table** for each field.

Fields

Select or clear a field to control how elements in an array or group field appear:

- To display each element in an array or group field in a single row, select the field.
- To display each element in an array or group field in a separate row, clear the field.

  Note: To display fields defined as a group field or as an array in the **Fields** list, select the **Groups and Arrays only** option. To display all fields in the record in the Fields list, clear this option. This option controls the fields that appear in the **Fields** list and in the **Table Properties** dialog box, but not the fields that appear in the **Database Row Test Output** window.

How do you want to handle multiple instances of selected records?

- **New Row**. A new row appears or is written to the target for every element in the array.
- **Ignore**. PowerExchange does not display, or write to the target, second and subsequent elements in the array.
- **Array**. PowerExchange displays, or writes to the target, the number of elements specified in the **Array** list in a single row of output.

PowerExchange populates the output row until it is full, and then completes one of the following actions:

- If you clear the **New Row on Overflow** option, PowerExchange ignores subsequent records or segments.
- If you select the **New Row on Overflow** option, PowerExchange displays a new row with the overflow records or segments.

Multiple Arrays in a Single Input Row

Generates multiple output rows from a single record that contains multiple arrays, or OCCURS clauses. PowerExchange sets the output fields to NULL when the data in the record is exhausted.

Enabled for a table with an imported COPYLIB with multiple OCCURS clauses.

Author

Diane Fleming